

Chapter 4. Custom Container Images





Outline

- Create a containerfile using basic commands.
- Create a containerfile using best practices.
- Run rootless containers with Podman.



Create Images with Containerfiles

- A **Containerfile/Dockerfile** lists a set of instructions to **construct a container image**. Once built, the image is used to create **any number of containers**.
- **Each instruction** causes **some change** that is captured in a **resulting image layer**. These layers are **stacked together** to form the **resulting container image**.

Choosing a Base Image

- When building the image, the **instructions** within the Containerfile are **run in order** and are **applied on top of a base image**. A **base image** is the image from which your Containerfile and its resulting image is built.
- The **base image** you choose determines the Linux distribution, as well as any or all of the following components:
 - Package manager ,Init system, Filesystem layout ,Preinstalled dependencies and runtimes
- The base image can also **influence image size, vendor support, and processor compatibility**, among other factors.



Choosing a Base Image

- When building the image, the **instructions** within the Containerfile are **applied on top of a base image**. A **base image** is the image from which your Containerfile and its resulting image is built.
- The **base image** you choose determines the Linux distribution, as well as any or all of the following components **such as Package manager**

```
# This is a comment line 1
FROM registry.redhat.io/ubi8/ubi:8.6 2
LABEL description="This is a custom httpd container image" 3
RUN yum install -y httpd 4
EXPOSE 80 5
ENV LogLevel "info" 6
ADD http://someserver.com/filename.pdf /var/www/html 7
COPY ./src/ /var/www/html/ 8
USER apache 9
ENTRYPOINT ["/usr/sbin/httpd"] 10
CMD ["-D", "FOREGROUND"] 11
```

```
Dockerfile
FROM Ubuntu
RUN apt-get update && apt-get -y install python
RUN pip install flask flask-mysql
COPY . /opt/source-code
ENTRYPOINT FLASK_APP=/opt/source-code/app.py flask
run
```



Containerfile Instructions

- **FROM** : Takes the name of the **base image** as an argument.
- **WORKDIR** : Sets the **current working directory within the container**. Later instructions run within this directory. If the WORKDIR doesn't exist, it will be created even if it's not used in any subsequent Dockerfile instruction.
- **COPY** : Copies files from the build host into the file system of the resulting container image. Relative paths respect the host's current working directory, known as the build context, and the working directory within the container as defined by WORKDIR. It **is not possible to copy a remote file by using its URL** with this Containerfile instruction.
- **ADD** : Copies files or folders from a **local or remote source** and adds them to the container's file system. If used to copy local files, those must be in the working directory. The ADD instruction also unpacks **local .tar archive** files to the destination image directory.
- **RUN** : Runs a command in the container and commits the resulting state of the container to a new layer within the image.
- **ENTRYPOINT** : Sets the executable to run when the container is started.
- **CMD** : Runs a command when the container is started. This command is passed to the executable defined by ENTRYPOINT. Base images define a default ENTRYPOINT, which is usually a shell executable, such as Bash.



Containerfile Instructions

- **USER** : Changes the active user within the container. Later instructions run as this user, including the CMD instruction. It is a good practice to define a different user other than root for security reasons.
- **LABEL** : Adds a key-value pair to the metadata of the image for organization and image selection.
- **EXPOSE** : Adds a port to the **metadata** for the image indicating that an **application within the container will bind to this port**. Note that this does not bind the port on the host and is for documentation purposes.
- **ENV** : Defines environment variables that are available in the container. You can declare multiple ENV instructions within the Containerfile. You can use the env command inside the container to view each of the environment variables.
- **ARG** : Defines **build-time variables**, typically to make a customizable container build. Developers commonly configure the ENV instructions by using the ARG instruction. This is useful for preserving the build-time variables for run time.
- **VOLUME** : Defines where to store data outside of the container. The value designates the path where Podman mounts the directory inside of the container. You can define more than one path to create multiple volumes.



Containerfile Instructions

- Each Containerfile instruction runs in an **independent container** using an intermediate image built from every previous command. This means each instruction is independent from other instructions in the Containerfile.

Container Image Tags

- When building a container image, you can specify an image name used to later identify the image. The image name is a string composed of letters, numbers, and some special characters.
- An image tag comes after the image name and is delimited by a colon (:). When an image tag is omitted, latest is used as the default.
- In many cases, you should **specify tags in addition to latest**. Because latest is the default tag applied to new images, references using the latest tag can change unintentionally.
- The full name of the image includes **both a name and an optional image tag**.
- For example, in the full image name my-app:1.0, the name is my-app and the tag is 1.0.
- **podman build -t <image-name>:<tag> .** When Containerfile exists in your current working directory.

Example Containerfile for building a simple Apache web server container



```
# This is a comment line ❶
FROM registry.redhat.io/ubi8/ubi:8.6 ❷
LABEL description="This is a custom httpd container image" ❸
RUN yum install -y httpd ❹
EXPOSE 80 ❺
ENV LogLevel "info" ❻
ADD http://someserver.com/filename.pdf /var/www/html ❼
COPY ./src/ /var/www/html/ ❽
USER apache ❾
ENTRYPOINT ["/usr/sbin/httpd"] ❿
CMD ["-D", "FOREGROUND"] ⓫
```

- ❶ Lines that begin with a hash, or pound, sign (#) are comments.
- ❷ The FROM instruction declares that the new container image extends the registry.redhat.io/ubi8/ubi:8.6 container base image.
- ❸ The LABEL instruction is responsible for adding generic metadata to an image.
- ❹ RUN executes commands in a new layer on top of the current image. The shell that is used to execute commands is /bin/sh.
- ❺ EXPOSE indicates that the container listens on the specified network port at runtime. The EXPOSE instruction defines metadata only; it does not make ports accessible from the host.
- ❻ ENV is responsible for defining environment variables that are available in the container.
- ❼ The ADD instruction copies files or folders from a local or remote source and adds them to the container's file system.
- ❽ COPY copies files from the working directory and adds them to the container's file system.
- ❾ USER specifies the username or the UID to use when running the container image for the RUN, CMD, and ENTRYPOINT instructions
- ❿ ENTRYPOINT specifies the default command to execute when the image runs in a container.
- ⓫ CMD provides the default arguments for the ENTRYPOINT instruction.



Questions

- Difference between copy and add (with example).

Local only	Remote url pr tar file
------------	------------------------

- Difference between cmd and entrypoint (with example).

override	append
----------	--------

- Inspect the number of image layers in the hello-server:bad image.

podman image tree hello-server:bad

Section 4.3: Build Images with Advanced Containerfile Instructions





Advanced Containerfile Instructions

- **Developers commonly create containers that have a number of downsides, for example:**
 - 1) The container is bound to a specific environment, and requires a rebuild before deploying to a production environment.
 - 2) The container contains developer tools, such as a debugger, text editors, or compilers.
 - 3) The container contains build-time dependencies that are not necessary at runtime.
 - 4) The container generates a large volume of files stored on the copy-on-write file system, which limits the performance of the application.
- **This section focuses on advanced container patterns that help you limit previously mentioned issues, such as:**
 - 1) Reducing image storage footprint by using the multistage container build pattern.
 - 2) Customizing container runtime with environment variables.
 - 3) Using volumes to decrease the container size and increase the performance of writing files.



ENV Instruction

- The ENV instruction lets you **specify environment dependent configuration**, for example, hostnames, ports or usernames.
- Then the **containerized application** can **pick these values as environment variables** and use them accordingly.
- To **include an environment variable in Containerfile**, use the key=value format.
 - **ENV DB_HOST="database.example.com"**
- Then you can **retrieve the environment variable in your application**. For example, the following snippet is a Python script that retrieves the DB_HOST environment variable.

```
from os import environ

DB_HOST = environ.get('DB_HOST')

# Connect to the database at DB_HOST...
```



ARG Instruction

- Use the ARG instruction to **define build-time variables**, typically to make a customizable container build.
- You can configure a **default build-time variable value** if the developer **does not provide it at build time**. Use the following syntax to define a build-time variable:
 - ARG key[=default value]
- When you build the container image, use the **--build-arg** flag to set the value, such as **podman build --build-arg key=example-value** for the preceding example.
- If you **do not provide the --build-arg flag**, then Podman **uses the default values during the build process**. However, using the ARG instruction **means that you can change the values during the build process without changing the Containerfile**.

```
ARG VERSION="1.16.8" \  
    BIN_DIR=/usr/local/bin/  
  
RUN curl "https://dl.example.io/${VERSION}/example-linux-amd64" \  
    -o ${BIN_DIR}/example
```



ENV & ARG Instruction

- Developers commonly **configure the ENV instructions by using the ARG instruction**. This is useful for **preserving the build-time variables for run time**.

```
ARG VERSION="1.16.8" \  
    BIN_DIR=/usr/local/bin/  
  
ENV VERSION=${VERSION} \  
    BIN_DIR=${BIN_DIR}  
  
RUN curl "https://dl.example.io/${VERSION}/example-linux-amd64" \  
    -o ${BIN_DIR}/example
```

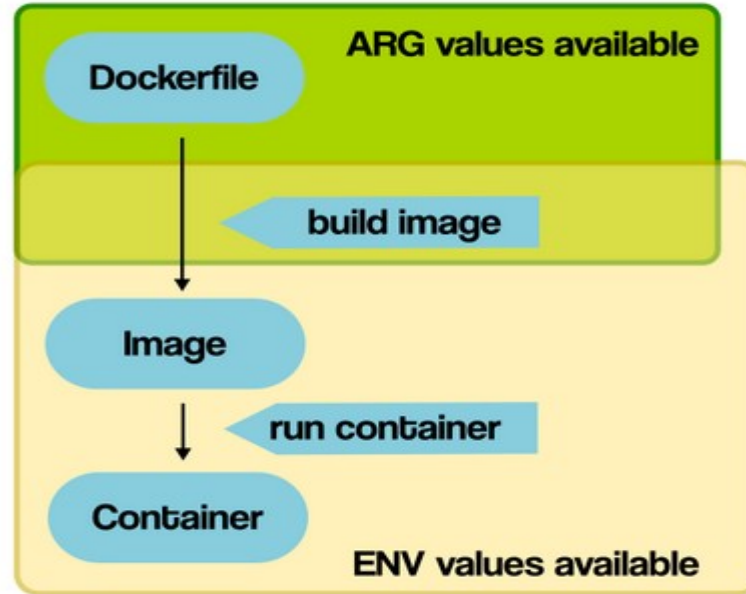
- In the preceding example, **ENV instruction uses the value of the ARG instruction to configure the run-time environment variables**.
- If you **do not configure the ARG default values** but you **must configure the ENV default values**, then **use the \${VARIABLE:-DEFAULT_VALUE} syntax**, such as:

```
ARG VERSION \  
    BIN_DIR  
  
ENV VERSION=${VERSION:-1.16.8} \  
    BIN_DIR=${BIN_DIR:-/usr/local/bin/  
  
RUN curl "https://dl.example.io/${VERSION}/example-linux-amd64" \  
    -o ${BIN_DIR}/example
```



ENV & ARG Instruction

- The **ARG** arguments are **only available when building the image**; they are unavailable once you have built the image. The running containers cannot access ARG values.
- While **ENV** parameters are **available to the application containers during build and when the container is running**.





ENV & ARG Instruction

- ARG is used to define **build-time** variables, whereas ENV is used to define **run-time** variables.
- ARG is **not available** in the final image, whereas ENV is **set in the final image**.
- ARG can be **overridden during the build process** using the -- build-arg flag, whereas ENV **cannot be overridden during the build process**.
- ARG is used to customize the build process, whereas ENV is used to configure the container environment.
- ARG is typically used for passing build-time arguments to Dockerfile, whereas ENV is used for setting environment variables, defining the working directory, and setting user.



VOLUME Instruction

- Use the VOLUME instruction **to store data outside of the container**. The value is the path where Podman mounts the directory inside of the container. The VOLUME instruction accepts more than one path to create multiple volumes.

```
FROM registry.redhat.io/rhel9/postgresql-13:1  
  
VOLUME /var/lib/pgsql/data
```

- In the previous Containerfile, **if you add instructions that update /var/lib/pgsql/data after the VOLUME instruction**, then those instructions are ignored.
- To retrieve the local directory used by a volume, you can use the **podman inspect VOLUME_ID** command.

```
[user@host ~]$ podman inspect VOLUME_ID  
[  
  {  
    "Name": "VOLUME_ID",  
    "Driver": "local",  
    "Mountpoint": "/home/your-name/.local/share/containers/storage/volumes/VOLUME_ID/_data",  
    ...output omitted...  
    "Anonymous": true  
  }  
]
```



VOLUME Instruction

- The **Mountpoint** field gives you the **absolute path** to the directory where the volume exists on **your host file system**.
- Volumes **created from the VOLUME instruction** have a **random ID** in the **Name** field and are considered **Anonymous volumes**.
- Podman ties the **lifecycle of the anonymous volume to the lifecycle of the container**. The **volume is created and removed with the container**.
- To create a volume that **does not depend on a container lifecycle** you must create a **named volume**.
 - `podman volume create VOLUME_NAME`
- `podman volume ls --format="{{.Name}}\t{{.Mountpoint}}"` command to **include the Mountpoint field of every volume**.

```
[user@host ~]$ podman volume ls \
--format="{{.Name}}\t{{.Mountpoint}}"
0a8c...82c2  /home/your-name/.local/share/containers/storage/volumes/0a8c...82c2/_data
252d...b2ed  /home/your-name/.local/share/containers/storage/volumes/252d...b2ed/_data
```



ENTRYPOINT Instruction

- The ENTRYPOINT and CMD instructions **specify the command to execute when the container starts**. A valid Containerfile must have **at least one of these instructions**. It can also have both.
- The ENTRYPOINT instruction **defines an executable, or command, that is always part of the container execution**. This means that additional arguments are passed to the provided command **(append)**.

Consider the following example:

```
FROM registry.access.redhat.com/ubi8/ubi-minimal:8.5
ENTRYPOINT ["echo", "Hello"]
```

Running a container from the previous Containerfile with no arguments prints "Hello".

```
[user@host ~]$ podman run my-image
Hello
```

If you provide the container with the "Red" and "Hat" arguments, then it prints "Hello Red Hat".

```
[user@host ~]$ podman run my-image Red Hat
Hello Red Hat
```



CMD Instruction

- If you **only use the CMD instruction**, then passing arguments to the container **overrides** the command provided in the CMD instruction.
- If you list more than one CMD then **only the last CMD will take effect**.

```
FROM registry.access.redhat.com/ubi8/ubi-minimal:8.5
CMD ["echo", "Hello", "Red Hat"]
```

Running a container from the previous Containerfile with no arguments prints Hello Red Hat.

```
[user@host ~]$ podman run my-image
Hello Red Hat
```

Running a container with the argument `whoami` overrides the echo command.

```
[user@host ~]$ podman run my-image whoami
root
```



ENTRYPOINT and CMD Instructions

- When a Containerfile specifies both **ENTRYPOINT** and **CMD** then CMD changes its behavior. In this case the values provided to CMD are passed as default arguments to the ENTRYPOINT.

```
FROM registry.access.redhat.com/ubi8/ubi-minimal:8.5
ENTRYPOINT ["echo", "Hello"]
CMD ["Red", "Hat"]
```

The previous command prints Hello Red Hat when the container runs.

```
[user@host ~]$ podman run my-image
Hello Red Hat
```

If you provide the argument Podman to the container, then it prints Hello Podman.

```
[user@host ~]$ podman run my-image Podman
Hello Podman
```

Formats of ENTRYPOINT and CMD instructions



Text array [Exec Form]

- The executable takes the form of a text array, such as: `ENTRYPOINT ["executable", "param1", ..., "paramN"]`
- The exec form is parsed as a JSON array, which means that you must use double-quotes (") around words not single-quotes (').

String form [Shell Form]

- The command and parameters are written in a text form, such as: `CMD executable param1 ... paramN`
- Unlike the shell form, the **exec form does not invoke a command shell**. This means that normal shell processing does not happen.
- For example, `CMD ["echo", "$HOME"]` **will not do variable substitution** on \$HOME
- If you want shell processing then either use the **shell form** or **execute a shell directly**, for
- Example: `CMD ["sh", "-c", "echo $HOME"]`. When using the exec form and executing a shell directly, as in the case for the shell form, it is the shell that is doing the environment variable expansion, not docker



Podman Secret

- A secret is a blob of **sensitive information** required by containers at runtime.
- This can be usernames, passwords, or keys. For example, if your containerized application requires credentials for connecting to a database, then you must store those credentials as a secret.
- After creating the secret, you must instruct the application container to make the credentials available to the application when it starts. To manage this process, use the podman secret subcommands.



Podman Secret Commands

- Create a secret named db secret
 - `echo "R3d4ht123" > dbsecretfile`
 - `podman secret create dbsecret dbsecretfile`
- List all secrets.
 - `Podman secret ls`
- Inspect the secret.
 - `Podman secret inspect dbsecret`
- Run a container and pass the secret.
 - `podman run -it --secret dbsecret --name myapp registry.access.redhat.com/ubi8/ubi /bin/bash`
 - `cat /run/secrets/dbsecret`



Multistage Builds

- A multistage build uses **several FROM instructions (called stages)** to create an independent build on every stage.
- Every stage might use a different base image (defined by the FROM instruction), and you can copy files among stages. **Only the dependencies declared in the last stage are included in the resulting image**
- Therefore you can **use previous stages to avoid including unnecessary intermediate layers in the resulting image.**
- One of the **main use cases for multistage builds** is to **reduce the image size by only keeping the necessary dependencies.**

Multistage Builds



- For example, consider the following example, where a Node application is containerized.

```
FROM node:12.13.0-alpine
WORKDIR /app
COPY . .
RUN npm install
RUN npm run build
RUN serve build
```

- The **npm install** command **installs all the required NPM packages**, which includes packages that are **only needed at build-time**. The **npm run build** creates an **optimized production-ready version of the application from the installed packages**. Then, you create a simple HTTP server to expose the application by using the **serve** command.
- If you build an image from the previous Containerfile, then the image **contains both the build-time dependencies and the runtime dependencies**, therefore increasing the image size.

Multistage Builds



- **To avoid this issue (increasing image size)**, you can copy the output of one or more stages into a final stage. you can define two stages:
 - 1) **First stage:** execute npm install to fetch the corresponding Node modules. Then, generate the optimized version of the application by using npm run build.
 - 2) **Second stage:** Copy the optimized version of the application and serve the static files by using an HTTP server, such as NGINX or Apache Server.
- The final container image consists of only the second stage, **therefore reducing its size.**

```
# First stage
FROM registry.access.redhat.com/ubi8/nodejs-14:1 as builder ❶
COPY ./ /opt/app-root/src/
RUN npm install
RUN npm run build ❷

# Second stage
FROM registry.access.redhat.com/ubi8/nodejs-14-minimal:1 ❸
COPY --from=builder /opt/app-root/src/ /opt/app-root/src/ ❹
EXPOSE 8080
CMD node dist/index.js
```

Reduce Image Layers



- You can reduce the **number of layers to one** by chaining the commands.
- In Linux, you can chain commands **by using the double ampersand (&&)**. You can **also use the backslash character (\)** to break a long command into multiple lines.
- The **advantage of chaining commands** is that you **create less container image layers**, which typically results in **smaller images**.

- **USE**

```
RUN mkdir /etc/gitea && \  
  chown root:gitea /etc/gitea && \  
  chmod 770 /etc/gitea
```

instead of

```
RUN mkdir /etc/gitea  
RUN chown root:gitea /etc/gitea  
RUN chmod 770 /etc/gitea
```

Reduce Image Layers



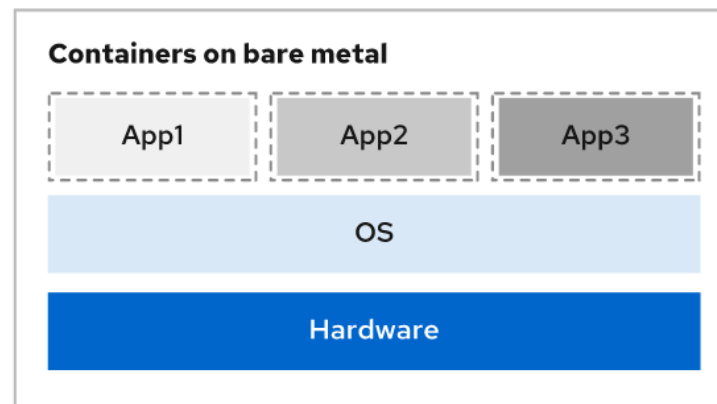
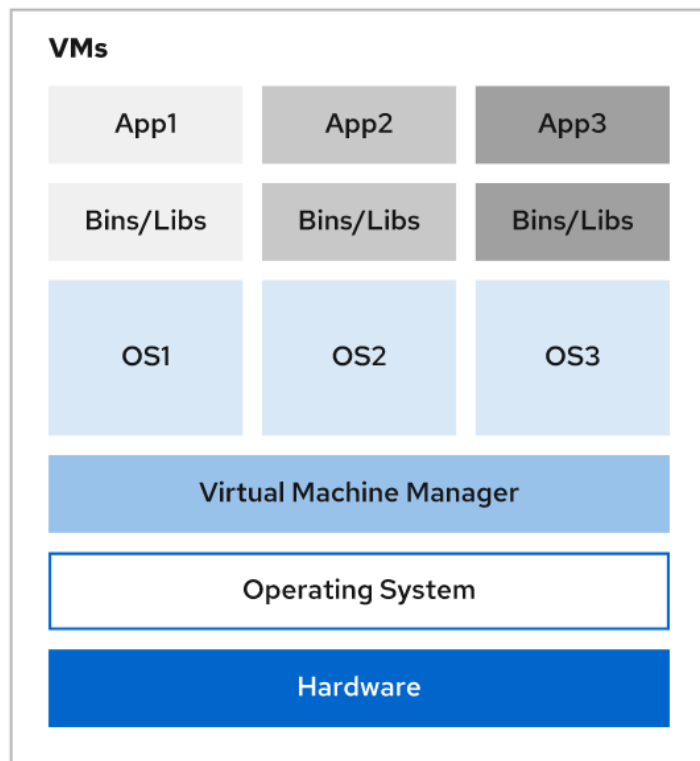
- However, chained commands are more difficult to debug and cache.
- Configure Podman to **squash** the layers. Use the **--squash** option to squash layers declared in the Containerfile. Alternatively, use the **--squash-all** option to also squash the layers from the parent image.
 - 1) **--squash**: Squash all of the image's new layers into a single new layer; **any preexisting layers are not squashed**.
 - 2) **--squash-all**: Squash all of the new image's layers (including those inherited from a base image) into a single new layer.

```
[user@host ~] $ podman build -t localhost/not-squashed .  
...output omitted...  
[user@host ~] $ podman build --squash -t localhost/squashed .  
...output omitted...  
[user@host ~] $ podman build --squash-all -t localhost/squashed .  
...output omitted...  
[user@host ~] $ podman images --format="{{.Names}}\t{{.Size}}"  
[localhost/not-squashed:latest] 419 MB ❶  
[localhost/squashed:latest] 419 MB ❷  
[localhost/squashed-all:latest] 394 MB ❸
```

Section 4.5: Rootless Podman



Container Workload Isolation



 Container

VM and Containers application isolation



Analyzing Rootless Containers

- Rootless containers, or unprivileged containers, are containers that **do not require administrator privileges**.
- **Using rootless containers means the following:**
 - 1) The containerized process does not use the root user, which is a special privileged user in Linux and UNIX systems. Such a user is for administrative purposes and has the ID 0.
 - 2) The **root user inside of the container** is not the **root user outside of the container**.
 - 3) The container runtime does not use the root user. For example, if your container runtime runs as the root user, then containers managed by such runtime are not rootless containers.
- Because Podman starts each container as a new process, the runtime does not require elevated privileges.
- Note : The Podman process exits after creating a container. Then the container process ID attaches to the systemd parent process ID.



Analyzing Rootless Containers

- `podman run -d --name nginx-cont nginx`
- `podman exec -it nginx-cont /bin/bash`

```
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS
marwa@marwa:~$ podman run -d --name nginx-cont nginx
ff19666e5ff1b18c560b499175526ae699f5b8ca0c6deefe62c52ecde17e8320
marwa@marwa:~$ podman exec -it nginx-cont /bin/bash
root@ff19666e5ff1:/# id
uid=0(root) gid=0(root) groups=0(root)
root@ff19666e5ff1:/#
```

- `ps -ef | grep -i nginx-cont`

```
marwa@marwa:~$ ps -ef | grep -i nginx-cont
marwa    40603   3190   0 01:00 ?        00:00:00 /usr/bin/common --api-version 1 -c ff19666e5ff1b18c560b499175526ae699f5b8ca0c6deefe62c52ecde17e8320 -u ff19666e5ff1b18c560b499175526ae699f5b8ca0c6deefe62c52ecde17e8320 -r /usr/bin/crun -b /home/marwa/.local/share/containers/storage/overlay-containers/ff19666e5ff1b18c560b499175526ae699f5b8ca0c6deefe62c52ecde17e8320/userdata -p /run/user/1000/containers/overlay-containers/ff19666e5ff1b18c560b499175526ae699f5b8ca0c6deefe62c52ecde17e8320/userdata/pidfile -n nginx-cont --exit-dir /run/user/1000/libpod/tmp/exits --full-attach -s -l journald --log-level warning --runtime-arg --log-format=json --runtime-arg --log --runtime-arg=/run/user/1000/containers/overlay-containers/ff19666e5ff1b18c560b499175526ae699f5b8ca0c6deefe62c52ecde17e8320/userdata/oci-log --common-pidfile /run/user/1000/containers/overlay-containers/ff19666e5ff1b18c560b499175526ae699f5b8ca0c6deefe62c52ecde17e8320/userdata/common.pid --exit-command /usr/bin/podman --exit-command-arg --root --exit-command-arg /home/marwa/.local/share/containers/storage --exit-command-arg --runroot --exit-command-arg /run/user/1000/containers --exit-command-arg --log-level --exit-command-arg warning --exit-command-arg --cgroup-manager --exit-command-arg systemd --exit-command-arg --tmpdir --exit-command-arg /run/user/1000/libpod/tmp --exit-command-arg --runtime --exit-command-arg crun --exit-command-arg --storage-driver --exit-command-arg overlay --exit-command-arg --events-backend --exit-command-arg journald --exit-command-arg container --exit-command-arg cleanup --exit-command-arg ff19666e5ff1b18c560b499175526ae699f5b8ca0c6deefe62c52ecde17e8320
```

Changing the Container User



- When you create a Containerfile, **the user tends to be root**. This is because you require elevated privileges for certain operations, such as installing packages or making configuration changes.

```
[user@host ~] $ podman run registry.access.redhat.com/ubi9/ubi id  
uid=0(root) gid=0(root) groups=0(root)
```

The following container image uses the root user to start an HTTP server:

```
FROM registry.access.redhat.com/ubi9/ubi  
  
CMD ["python3", "-m", "http.server"]
```

Changing the Container User



- This is a security risk, because an **attacker could exploit the application, get access to the container**, and exploit further vulnerabilities to escape from the containerized environment into the host system.

```
FROM registry.access.redhat.com/ubi9/ubi

RUN adduser \
  --no-create-home \
  --system \
  --shell /usr/sbin/nologin \
  python-server

USER python-server

CMD ["python3", "-m", "http.server"]
```

- Note that the **RUN instruction runs as the root user**, because it **precedes the USER instruction**.

Lab: section 4.7



Dockerfile Layered Architecture



Layered architecture

Dockerfile

```
FROM Ubuntu
```

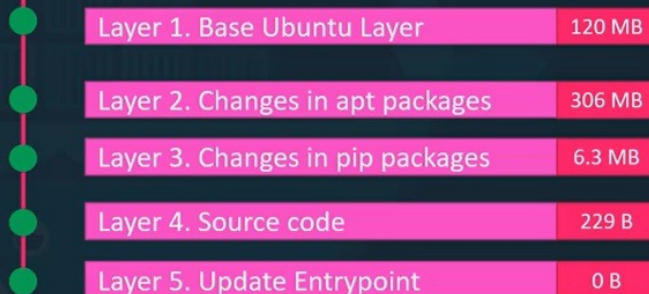
```
RUN apt-get update && apt-get -y install python
```

```
RUN pip install flask flask-mysql
```

```
COPY . /opt/source-code
```

```
ENTRYPOINT FLASK_APP=/opt/source-code/app.py flask  
run
```

```
docker build Dockerfile -t mmumshad/my-custom-app
```



Dockerfile2

```
FROM Ubuntu
```

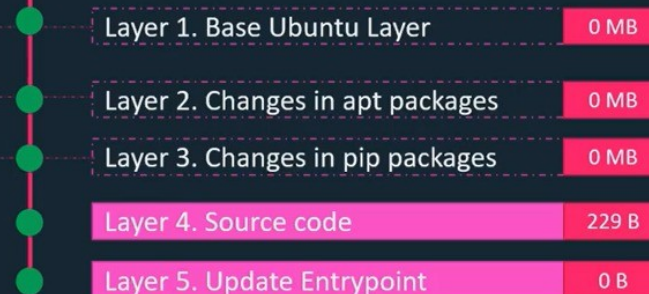
```
RUN apt-get update && apt-get -y install python
```

```
RUN pip install flask flask-mysql
```

```
COPY app2.py /opt/source-code
```

```
ENTRYPOINT FLASK_APP=/opt/source-code/app2.py flask  
run
```

```
docker build Dockerfile2 -t mmumshad/my-custom-app-2
```





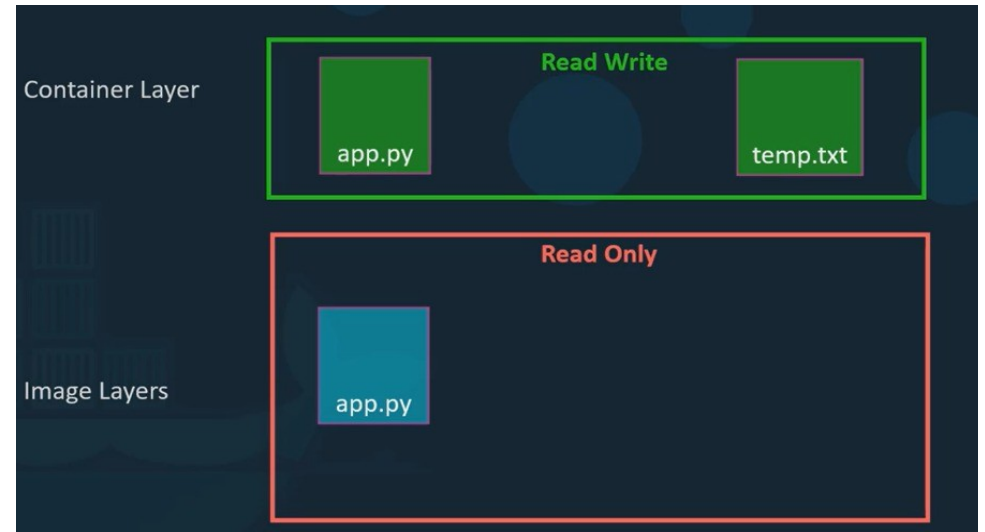
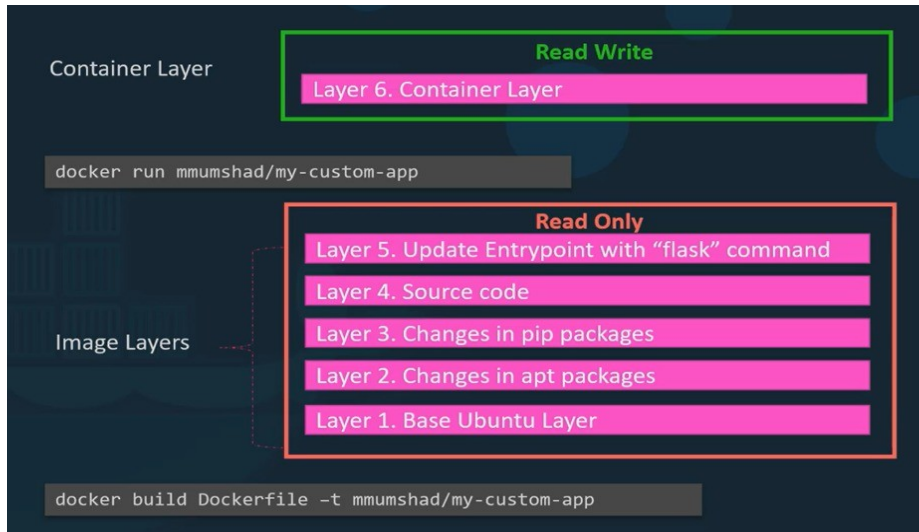
Dockerfile Layered Architecture

- A Docker image consists of **read-only layers** each of which represents a Dockerfile instruction. The **layers are stacked** and **each one is a delta of the changes from the previous layer**.
- The first three layers of both applications are the same. So, docker **is not going to build the first three layers again**.
- Instead it **reuses the same three layers it built for the first application from the cache**. And only create the last two layers with the new sources and the new entry point.
- By this way, Docker **builds images faster and efficiently saves disc space**.



Dockerfile Layered Architecture

- When you run an image and generate a container, you add a **new writable layer**, also called the container layer, on top of the underlying layers.
- All changes made to the running container, such as writing new files, modifying existing files, and deleting files, are written to this writable container layer.



Lab

